CHARGEPOINT

EV Charging Station Reservation and Billing System

DBMS Course Project - Final Design Document

Members:

Raghav Mithinti 2023A3PS0296H

Aaditya Rajput 2023A8PS0384H

Akhil Thindi 2023AAPS0172H

Samrat Chaturvedi 2023A7PS0125H

Kartikeya Srivastava 2023A3PS0385H

GitHub: https://github.com/Raghav50w/dbms_project

1. Problem statement, users and scope

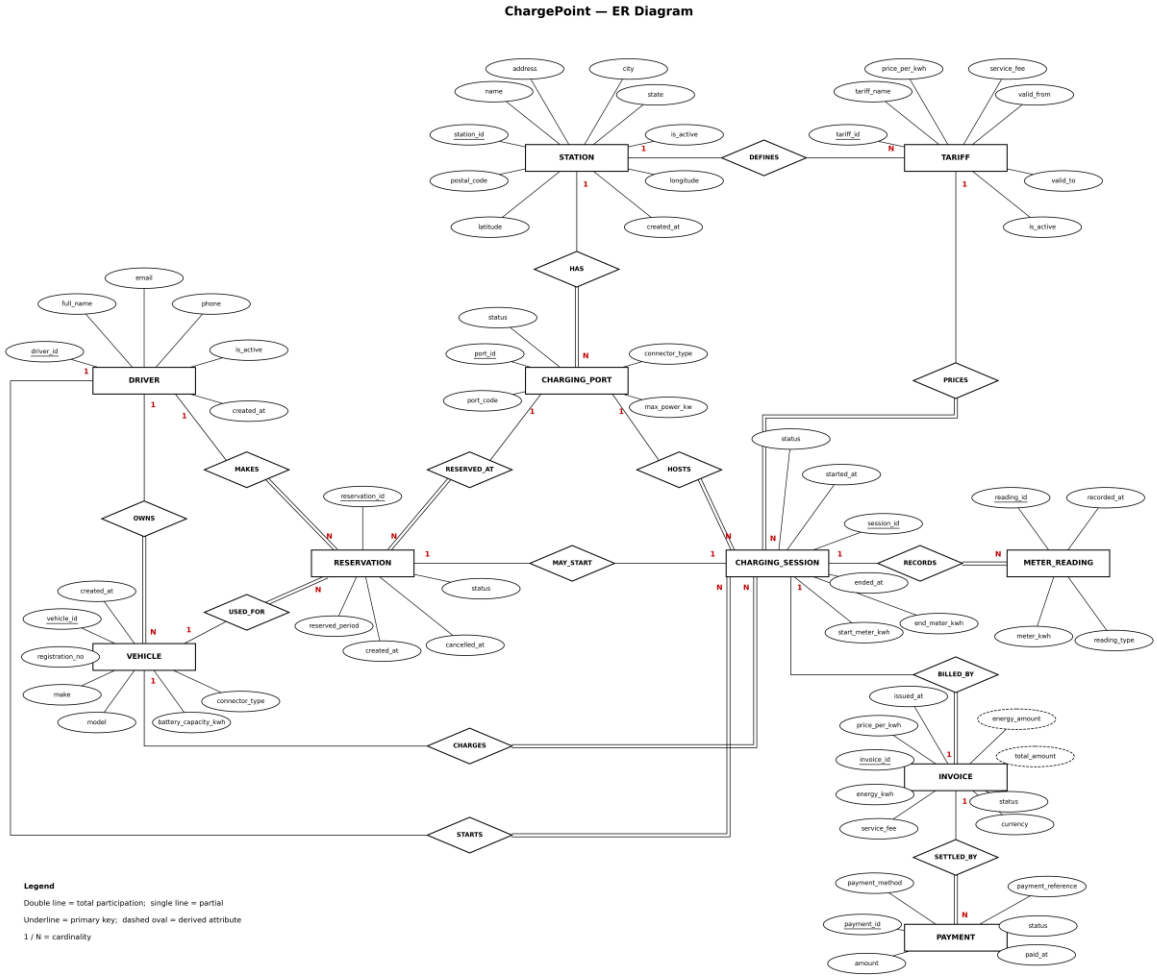
EV drivers need to find a compatible charging port, reserve a time slot, charge a vehicle and receive an accurate energy-based bill. Operators need to manage availability, tariffs, sessions, outages and revenue. ChargePoint uses PostgreSQL as the source of truth. This is an academic prototype: payments are simulated and physical chargers are not controlled.

Users: Drivers manage vehicles, reservations and invoices, operators manage port status/tariffs, sessions and reports, administrators maintain reference data.

Assumptions: timestamps use TIMESTAMPTZ, reservation ranges are half-open [start,end), energy = end meter - start meter, invoice = energy x price/kWh + service fee. Invoice price/fee are snapshots. Only successful payments count toward settlement.

2. ER/EER model and relationship cardinalities

Entities: DRIVERS, VEHICLES, STATIONS, CHARGING_PORTS, TARIFFS, RESERVATIONS, CHARGING_SESSIONS, METER_READINGS, INVOICES, PAYMENTS.



Relationship	**Cardinality / business rule**
Driver-Vehicle	1:N, each vehicle belongs to one driver
Station-Port / Station-Tariff	1:N for both relationships
Driver, Vehicle, Port-Reservation	Each reservation references one of each
Reservation-Session	Reservation-linked session is optional and at most one, walk-in sessions allowed
Session-Meter Reading	1:N, readings are timestamped
Session-Invoice	1:0..1, at most one invoice per session
Invoice-Payment	1:N, supports partial payments and failed attempts

Participation: total (double line), meaning every vehicle has a driver, every port belongs to a station, every reservation has a driver, vehicle and port, every session has a driver, vehicle, port and tariff, and every meter reading, invoice and payment has its parent. Partial (single line), meaning a reservation may never become a session (cancelled/no-show), a session may not yet have an invoice, and a tariff may be global (no station).

3. Relational schema and integrity constraints

- drivers(driver_id PK, full_name, email UNIQUE, phone UNIQUE, created_at, is_active)
- vehicles(vehicle_id PK, driver_id FK, registration_no UNIQUE, make, model, battery_capacity CHECK > 0, connector_type CHECK)
- stations(station_id PK, name, address, city, state, postal_code, coordinates CHECK, is_active)
- charging_ports(port_id PK, station_id FK, port_code, connector_type CHECK, max_power CHECK > 0, status CHECK, UNIQUE(station_id, port_code))
- tariffs(tariff_id PK, station_id FK, tariff_name, price_per_kwh CHECK >= 0, service_fee CHECK >= 0, validity)
- reservations(reservation_id PK, driver_id FK, vehicle_id FK, port_id FK, reserved_period, status CHECK)
- charging_sessions(session_id PK, reservation_id UNIQUE FK nullable, driver_id/vehicle_id/port_id/tariff_id FKs, meter readings, status)
- meter_readings(reading_id PK, session_id FK, recorded_at, meter_kwh CHECK >= 0, reading_type)
- invoices(invoice_id PK, session_id UNIQUE FK, energy, price snapshot, fee snapshot, total, status)
- payments(payment_id PK, invoice_id FK, amount CHECK > 0, method, reference UNIQUE, status)

Candidate keys: drivers.email, drivers.phone, vehicles.registration_no, charging_ports(station_id, port_code), charging_sessions.reservation_id, invoices.session_id, payments.payment_reference.

DBMS-level protections: PK/FK/UNIQUE/CHECK constraints, GiST exclusion constraint against overlapping active reservations per port, partial unique index for one active session per port, payment-status trigger, station-utilization view, targeted indexes.

4. Normalization and design justification

1NF: atomic fields, multiple readings, payments, vehicles and ports are separate rows. 2NF: single-column primary keys, non-key attributes depend on the key. 3NF: driver, station, port, tariff, session, invoice and payment facts live in separate relations, names/contact/address are not copied into reservation rows. Invoice price and fee are intentionally snapshotted so tariff changes do not rewrite historic bills.

Reservation overlap is enforced in PostgreSQL rather than only by a pre-check in the application, because concurrent requests can race. The range constraint is the final guard.

5. Application architecture and stack
Frontend: responsive HTML/CSS. Backend: Flask/Python. Database access: psycopg parameterized SQL. Database: PostgreSQL 15+. Tests: pytest. Payments and charger control are simulated/not integrated.

The system follows a three-tier architecture. The browser UI sends requests to Flask routes, which run parameterized SQL through psycopg against PostgreSQL. PostgreSQL enforces all business rules through constraints and transactions, keeps invoice payment status up to date with a trigger, provides station utilization through a view, and supports reporting queries with indexes.
The application has three pages: Stations & reservations (filter stations by city, reserve a port, cancel a confirmed reservation, delete a cancelled one), Sessions & payments (start and end sessions, generate invoices, record payments) and Reports (five reporting queries shown as tables).

6. End-to-end workflows
A - Reservation: driver chooses station/port/time, backend validates input and inserts reservation, PostgreSQL rejects overlaps, UI reports confirmation/conflict. Confirmed reservations can be cancelled and cancelled ones deleted.
B - Session and invoice: operator records start meter and tariff, later records end meter/time, a transaction closes the session and inserts invoice, end reading must be >= start reading.
C - Payment: driver records a simulated payment, trigger recalculates invoice state as unpaid, partially paid or paid from successful payments. Payments above the outstanding balance are rejected.
D - Reporting: operator reviews utilization, energy sold, revenue, outstanding balances and reservation issue rates using joins, aggregates, CTEs and a view.

7. SQL and DBMS features
Reporting queries cover station utilization, monthly revenue, port usage, driver spend, cancellation/no-show rate, CTE-based session duration, outstanding invoices, connector compatibility using NOT EXISTS, upcoming reservations, and tariff comparison. DBMS features include transactions, exclusion constraint, trigger, view, indexes and integrity constraints.

8. Progress and team contributions
Completed: requirements and ER design, PostgreSQL schema with constraints, indexes, view and trigger, 10 reporting queries, transaction scripts for reservation, session/invoice and payment, synthetic data generator (500+ rows), Flask application with all four workflows (reservation, session and invoice, payment, reporting), tests.

Member	Contribution
Raghav Mithinti	Requirements gathering, ER/EER diagrams and relational database design

Member	Contribution
Aaditya Rajput	SQL schema creation, constraints, relationships and seed/sample data
Akhil Thindi	Flask backend, routes, API handling and application logic
Samrat Chaturvedi	Frontend development (HTML/CSS/JavaScript), UI design and integration with Flask
Kartikeya Srivastava	Testing and debugging, test reports, project documentation and final presentation